Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I expect loops, conditionals, lists, classes, functions, and the majority of topics used to apply to this project. I want to improve my skills with drafter.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest will probably creating dataclasses and writing function headers, the hardest will probably drafter.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think the dataclasses should be implemented first, that is the baseline for creating all the variables that are needed for this project. I think the task that will require the most time will be creating the drafter interface, or creating opponents to play against.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

I could get distracted talking to my friends, in order to mitigate this I will go to the quiet study area in my dorm or go to the library. I do not require any additional resources at this time and I do not have any questions.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.





Chips:

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**It is a basic online poker game. You and 2 opponents will have chips and be able to raise, fold, and call. Whoever has the best poker hand at the end wins.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be	1. players: List[Player]. This represents the user and two computer opponents. *Modification*: Populated at game start, modified during betting rounds to update chip counts, current bets, and fold statuses. 2. Community_cards: List[str]. This represents the five cards placed in the middle *Modification*: Cleared at the start of each hand; appends 3 cards on the flop, 1 card on the turn, and 1 card on the river.

meaningfully modified throughout the site.	3. Pot: int. This represents total bets *Modification*: Increases as players call, raise, or post blinds; resets to 0 when awarded to the winning player at showdown. 4. Current_turn_index: int. An integer index representing whose turn it is *Modification*: Increments after every action; wraps back to 0 using modulo logic until the betting round closes.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	name (str): The display name of the player (e.g., "User", "Bot 1", "Bot 2"). ● *Modification*: Set once during initial state setup; remains read-only during gameplay. chips (int): The total remaining stack of chips available to bet. ● *Modification*: Decreased when placing bets/blinds; increased when winning a hand or reset via game restart. hole_cards (List[str]): The private 2-card hand dealt to the player. ● *Modification*: Cleared and reassigned 2 new card strings at the start of every hand. current_bet (int): The amount of chips placed into the active betting round by this player. ● *Modification*: Increases during fold/call/raise actions; resets to 0 when transitioning to the next street (e.g., Flop to Turn). is_folded (bool): Flag indicating whether the player has surrendered their hand. ● *Modification*: Defaults to False; flips to True when the player selects "Fold" or when an AI opponent folds.

Routes (minimum 7) The names of your route functions	• index — Renders the main game table and controls. • start_new_game — Resets player stacks, clears community cards, and sets up a fresh deck. • deal_hand — Deals 2 hole cards to each player and posts initial small/big blinds. • player_action — Handles human actions (fold, call, raise) submitted via the user interface. • trigger_bot_turns — Executes decision-making logic for AI opponents when current_turn_index belongs to a bot. • advance_street — Transitions the game phase (Pre-flop → Flop → Turn → River → Showdown) and deals community cards. • reset_stack — Recharges the human user's chip stack if they run out of money.
Pages (minimum 5) A brief description of the pages in your site	• Welcome / Setup Page: Allows the player to enter their name, select a starting chip stack size, and choose the difficulty level of the two AI opponents. • Main Poker Table Page: The primary screen showing community cards, the pot, the user's hand, AI card backs, player chip counts, and action buttons. • Showdown / Hand Results Page: Highlights the winning player, displays all opponent hole cards, reveals winning 5-card hand rankings, and awards the pot. • Game Over / Rebuy Page: Displayed when the user runs out of chips, featuring statistics for the session and a button to re-buy or start over. • Rules & Hand Rankings Guide: A reference page explaining Texas Hold'em hand hierarchies (High Card to Royal Flush) and basic game rules.
User Input (minimum 3)	Action Choice (SelectBox): • *Purpose*: Allows the player to select their main

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	action during their turn (Check/Call, Raise, Fold). Raise Amount (TextBox): ● *Purpose*: Accepts a numeric entry for the custom chip amount the user wants to raise when Raise is selected. Auto-Muck Losing Hands (CheckBox): ● *Purpose*: A toggle allowing the user to automatically hide/muck their losing hand at showdown rather than revealing it to opponents.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Validating Player Raise Amount: ● *Logic Evaluated*: Checks if the entered raise amount is both greater than the current minimum raise and less than or equal to the player's available chips. ● *Site Impact*: If valid, deducts chips and advances the turn; if invalid, prevents submission and displays an error message on the table page. Determining Hand Advancement vs. Showdown: ● *Logic Evaluated*: Checks if all non-folded players have matched the highest current_bet in the active round. ● *Site Impact*: If true and less than 5 community cards are present, invokes advance_street. If 5 community cards are out, routes the site to the Showdown Page. Evaluating Uncontested Winner (Fold Detection): ● *Logic Evaluated*: Evaluates whether the number of active players with is_folded == False equals 1. ● *Site Impact*: Skips remaining community card deals immediately, awards the pot to the last

	remaining player, and redirects to the Hand Results Page.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	• Description & Purpose: This loop iterates through the list of Player instances stored in state.players to evaluate if every active player has matched the current highest bet or gone all-in. • Why It Is Meaningful: It governs core Texas Hold'em game flow. Without iterating through state.players, the app cannot determine when a betting round is legally finished, preventing unauthorized state progression to the Flop, Turn, River, or Showdown.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I said: *So I am designing a texas holdem game with two opponents in python. generate me: "State Dataclass Attributes*

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site." "Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)" "Routes (minimum 7)

The names of your route functions" "Pages (minimum 5)

A brief description of the pages in your site" "User Input (minimum 3)

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

For more information on these input types, see Drafter Documentation." "If Statements (minimum 3)

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site" "Loop (minimum 1)

This loop must iterate over the list/dict of dataclass instances in your State attribute.

Description of this loop and why it is meaningful to the site"

Gemini Responded with: *Here is a complete architectural design for your 3-player Texas Hold'em web application, structured to meet all your project requirements.*

State Dataclass Attributes

The `State` dataclass manages the global game session, player turns, and community board state.

Python

```
None
from dataclasses import dataclass, field
from typing import List

@dataclass
class State:
    players: List[Player] = field(default_factory=list)
    community_cards: List[str] = field(default_factory=list)
    pot: int = 0
    current_turn_index: int = 0
```

- *`players (List[Player])`: A list of `Player` instances representing the human*

user and the two AI opponents.
    - *Modification: Populated at game start, modified during betting rounds to update chip counts, current bets, and fold statuses.*
- *`community_cards` (`List[str]`): A list of card strings representing the shared board (Flop, Turn, River).*
    - *Modification: Cleared at the start of each hand; appends 3 cards on the flop, 1 card on the turn, and 1 card on the river.*
- *`pot` (`int`): The total amount of chips wagered in the current hand.*
    - *Modification: Increases as players call, raise, or post blinds; resets to 0 when awarded to the winning player at showdown.*
- *`current_turn_index` (`int`): An integer index tracking whose turn it is to act (0 for User, 1 for Bot 1, 2 for Bot 2).*
    - *Modification: Increments after every action; wraps back to 0 using modulo logic until the betting round closes.*

Second Dataclass: Player

The `Player` dataclass stores individual data for each participant at the table.

Python

```
None
@dataclass
class Player:
    name: str
    chips: int
    hole_cards: List[str] = field(default_factory=list)
    current_bet: int = 0
    is_folded: bool = False
```

- *`name` (`str`): The display name of the player (e.g., "User", "Bot 1", "Bot 2").*
    - *Modification: Set once during initial state setup; remains read-only during gameplay.*
- *`chips` (`int`): The total remaining stack of chips available to bet.*
    - *Modification: Decreased when placing bets/blinds; increased when winning a hand or reset via game restart.*
- *`hole_cards` (`List[str]`): The private 2-card hand dealt to the player.*
    - *Modification: Cleared and reassigned 2 new card strings at the start of every hand.*

- *`current_bet` (`int`): The amount of chips placed into the active betting round by this player.*
    - *Modification: Increases during fold/call/raise actions; resets to 0 when transitioning to the next street (e.g., Flop to Turn).*
- *`is_folded` (`bool`): Flag indicating whether the player has surrendered their hand.*
    - *Modification: Defaults to `False`; flips to `True` when the player selects "Fold" or when an AI opponent folds.*

Routes (7 Minimum)

1. *`index` — Renders the main game table and controls.*
2. *`start_new_game` — Resets player stacks, clears community cards, and sets up a fresh deck.*
3. *`deal_hand` — Deals 2 hole cards to each player and posts initial small/big blinds.*
4. *`player_action` — Handles human actions (`fold`, `call`, `raise`) submitted via the user interface.*
5. *`trigger_bot_turns` — Executes decision-making logic for AI opponents when `current_turn_index` belongs to a bot.*
6. *`advance_street` — Transitions the game phase (Pre-flop → Flop → Turn → River → Showdown) and deals community cards.*
7. *`reset_stack` — Recharges the human user's chip stack if they run out of money.*

Pages (5 Minimum)

1. *Welcome / Setup Page: Allows the player to enter their name, select a starting chip stack size, and choose the difficulty level of the two AI opponents.*
2. *Main Poker Table Page: The primary screen showing community cards, the pot, the user's hand, AI card backs, player chip counts, and action buttons.*
3. *Showdown / Hand Results Page: Highlights the winning player, displays all opponent hole cards, reveals winning 5-card hand rankings, and awards the pot.*
4. *Game Over / Rebuy Page: Displayed when the user runs out of chips, featuring statistics for the session and a button to re-buy or start over.*
5. *Rules & Hand Rankings Guide: A reference page explaining Texas Hold'em hand hierarchies (High Card to Royal Flush) and basic game rules.*

User Input Components (3 Minimum)

1. *Action Choice (`SelectBox`):*

    - *Purpose: Allows the player to select their main action during their turn (`Check/Call`, `Raise`, `Fold`).*
2. *Raise Amount (`TextBox`):*
    - *Purpose: Accepts a numeric entry for the custom chip amount the user wants to raise when `Raise` is selected.*
3. *Auto-Muck Losing Hands (`CheckBox`):*
    - *Purpose: A toggle allowing the user to automatically hide/muck their losing hand at showdown rather than revealing it to opponents.*

If Statements (3 Minimum)

1. *Validating Player Raise Amount:*
    - *Logic Evaluated: Checks if the entered raise amount is both greater than the current minimum raise and less than or equal to the player's available `chips`.*
    - *Site Impact: If valid, deducts chips and advances the turn; if invalid, prevents submission and displays an error message on the table page.*
2. *Determining Hand Advancement vs. Showdown:*
    - *Logic Evaluated: Checks if all non-folded players have matched the highest `current_bet` in the active round.*
    - *Site Impact: If true and less than 5 community cards are present, invokes `advance_street`. If 5 community cards are out, routes the site to the Showdown Page.*
3. *Evaluating Uncontested Winner (Fold Detection):*
    - *Logic Evaluated: Evaluates whether the number of active players with `is_folded == False` equals 1.*
    - *Site Impact: Skips remaining community card deals immediately, awards the pot to the last remaining player, and redirects to the Hand Results Page.*

Loop

Iterating Over State `players`

Python

```
None
def check_round_bets_complete(state: State) -> bool:
    highest_bet = max(player.current_bet for player in
state.players)
```

```
    for player in state.players:
        if not player.is_folded:
            if player.current_bet < highest_bet and
player.chips > 0:
                return False
    return True
```

- *Description & Purpose: This loop iterates through the list of `Player` instances stored in `state.players` to evaluate if every active player has matched the current highest bet or gone all-in.*
- *Why It Is Meaningful: It governs core Texas Hold'em game flow. Without iterating through `state.players`, the app cannot determine when a betting round is legally finished, preventing unauthorized state progression to the Flop, Turn, River, or Showdown.*

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.